

Fundamentos de Inteligencia Artificial

Teoría, componentes, justificación y flujos de implementación en Python

Material estructurado a partir de seis infografías

Aprendizaje supervisado • Aprendizaje por refuerzo • Deep Learning
IA generativa • IA simbólica

Septiembre de 2026

Índice general

Cómo utilizar este documento	3
1. Aprendizaje supervisado: de los datos etiquetados a la predicción	4
1.1. Fundamentos teóricos	4
1.2. Componentes, definiciones, uso y justificación	5
1.3. Flujo teórico	6
1.4. Flujo de programación en Python	6
1.5. Aspectos teóricos que deben controlarse	6
2. Aprendizaje por refuerzo: interacción, decisiones y recompensas	8
2.1. Fundamentos teóricos	8
2.2. Componentes, definiciones, uso y justificación	9
2.3. Flujo de programación en Python	9
2.4. Justificación de la interacción	10
3. Aprendizaje supervisado: pipeline de datos, modelo y predicción	11
3.1. Fundamentos teóricos	11
3.2. Componentes, definiciones, uso y justificación	12
3.3. Implementación	12
4. Deep Learning: representación jerárquica y optimización	13
4.1. Fundamentos teóricos	13
4.2. Componentes, definiciones, uso y justificación	14
4.3. Flujo de programación con PyTorch	15
4.4. Relación entre profundidad y representación	15
5. IA generativa: modelado de distribuciones y generación de contenido	16
5.1. Fundamentos teóricos	16
5.2. Componentes, definiciones, uso y justificación	17
5.3. Flujo de programación	17
5.4. Aspectos conceptuales	18
6. IA simbólica: conocimiento explícito y razonamiento	19
6.1. Fundamentos teóricos	19
6.2. Componentes, definiciones, uso y justificación	20
6.3. Flujo de programación	20
6.4. Ventajas y límites conceptuales	21
7. Relación entre los paradigmas	22
7.1. Comparación conceptual	22
7.2. Patrón común de ingeniería	22
7.3. Arquitectura Python orientativa	23
7.4. Conclusión	23

A. Recursos y repositorios de aprendizaje	24
A.1. Repositorio de recursos sobre Inteligencia Artificial	24
A.1.1. Mejores repositorios GitHub para aprender IA	24
A.2. Microsoft – Generative AI for Beginners	24
A.3. Curso Básico de IA – Eneas Carneiro	25
A.4. Cursos de Python – Instituto Humai	25
A.5. Mapa de correspondencia con los capítulos	25
A.6. Referencias web	26

Cómo utilizar este documento

Cada capítulo corresponde a una de las imágenes proporcionadas. La imagen no se trata únicamente como una ilustración, sino como un **modelo funcional** que se descompone en componentes conceptuales, datos, procesos y resultados.

En cada capítulo se incluyen:

1. explicación teórica del paradigma;
2. descripción de los componentes representados;
3. relaciones entre los componentes;
4. una sección específica en formato `longtable`;
5. definición, uso y justificación de cada componente;
6. formulación matemática cuando resulta útil;
7. flujo de programación en Python;
8. criterios de diseño y errores habituales.

La columna **Justificación** de las tablas explica por qué el componente es necesario y qué problema resuelve dentro del paradigma, no solamente qué significa.

Capítulo 1

Aprendizaje supervisado: de los datos etiquetados a la predicción



Figura 1.1: Flujo del aprendizaje supervisado.

1.1. Fundamentos teóricos

El aprendizaje supervisado es un paradigma en el que un algoritmo aprende una relación entre una entrada X y una salida conocida y . El conjunto de entrenamiento está formado por pares

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n.$$

El objetivo es encontrar una función f_θ parametrizada por θ que produzca predicciones $\hat{y}_i = f_\theta(x_i)$ suficientemente próximas a las observaciones reales.

El entrenamiento puede expresarse como un problema de optimización:

$$\theta^* = \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(y_i, f_{\theta}(x_i)).$$

En regresión, y suele ser continuo; en clasificación, y pertenece a un conjunto discreto de clases. La separación entre entrenamiento y prueba permite estimar la capacidad de generalización del modelo sobre observaciones no utilizadas para ajustar sus parámetros.

1.2. Componentes, definiciones, uso y justificación

Cuadro 1.1: Componentes funcionales del aprendizaje supervisado.

Componente	Definición	Uso	Justificación
Datos etiquetados	Conjunto de pares (X, y) donde la salida correcta está disponible.	Entrenar y evaluar modelos supervisados.	Sin una referencia y no puede calcularse directamente el error supervisado que guía el ajuste.
Características X	Variables que describen cada observación.	Representar la información disponible para realizar una predicción.	El modelo necesita una representación numérica o estructurada de los factores explicativos.
Etiqueta y	Variable objetivo que se desea predecir.	Define el problema de aprendizaje.	Permite comparar predicción y realidad y convertir esa diferencia en una señal de aprendizaje.
Limpieza	Tratamiento de ausencias, inconsistencias, duplicados y valores extremos.	Preparar datos fiables.	Los errores de calidad pueden ser aprendidos por el modelo como si fueran patrones reales.
Transformación	Operaciones como escalado, codificación o transformación matemática.	Adaptar la representación a las hipótesis del algoritmo.	Algunos algoritmos son sensibles a escalas, categorías o distribuciones de las variables.
Selección de características	Elección de un subconjunto de variables relevantes.	Reducir dimensionalidad y ruido.	Puede disminuir complejidad, mejorar interpretabilidad y evitar variables irrelevantes.
Entrenamiento	Proceso de ajuste de los parámetros del modelo usando datos de entrenamiento.	Encontrar θ^* .	Es el mecanismo mediante el cual los ejemplos se convierten en conocimiento paramétrico.
Modelo	Función aprendida que transforma entradas en predicciones.	Inferir valores para nuevos ejemplos.	Constituye el artefacto reutilizable que encapsula el patrón aprendido.

Continúa en la siguiente página

Componente	Definición	Uso	Justificación
Datos nuevos	Observaciones cuya etiqueta puede no conocerse durante la inferencia.	Obtener predicciones.	Permiten aplicar el conocimiento aprendido fuera del conjunto de entrenamiento.
Evaluación	Comparación entre predicciones y valores reales mediante métricas.	Medir generalización.	Permite cuantificar errores y detectar modelos que memorizan el entrenamiento.

1.3. Flujo teórico

$(X, y) \rightarrow \text{preprocesamiento} \rightarrow \text{partición} \rightarrow \text{entrenamiento} \rightarrow f_{\theta^*} \rightarrow \hat{y}.$

1.4. Flujo de programación en Python

```

1 import pandas as pd
2 import numpy as np
3 from sklearn.model_selection import train_test_split
4 from sklearn.pipeline import Pipeline
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.ensemble import RandomForestRegressor
7 from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
8
9 df = pd.read_csv("datos.csv")
10
11 X = df[["x1", "x2", "x3"]]
12 y = df["y"]
13
14 X_train, X_test, y_train, y_test = train_test_split(
15     X, y, test_size=0.20, random_state=42
16 )
17
18 model = Pipeline([
19     ("scaler", StandardScaler()),
20     ("regressor", RandomForestRegressor(
21         n_estimators=200, random_state=42
22     ))
23 ])
24
25 model.fit(X_train, y_train)
26 y_pred = model.predict(X_test)
27
28 rmse = np.sqrt(mean_squared_error(y_test, y_pred))
29 mae = mean_absolute_error(y_test, y_pred)
30 r2 = r2_score(y_test, y_pred)
31
32 print(rmse, mae, r2)

```

1.5. Aspectos teóricos que deben controlarse

- **Sobreajuste:** el modelo aprende particularidades del entrenamiento.

- **Subajuste:** el modelo no captura la estructura relevante.
- **Fuga de información:** información del conjunto de prueba entra indirectamente en el entrenamiento.
- **Desbalance:** determinadas clases pueden estar infrarrepresentadas.
- **Generalización:** la métrica debe estimarse sobre datos independientes o mediante validación adecuada.

Capítulo 2

Aprendizaje por refuerzo: interacción, decisiones y recompensas



Figura 2.1: Ciclo agente-entorno del aprendizaje por refuerzo.

2.1. Fundamentos teóricos

El aprendizaje por refuerzo estudia cómo un agente toma decisiones secuenciales al interactuar con un entorno. En un modelo de decisión de Markov, el estado futuro depende del estado actual y de la acción, bajo la hipótesis de Markov:

$$P(s_{t+1}|s_t, a_t, \dots) = P(s_{t+1}|s_t, a_t).$$

El agente intenta maximizar el retorno:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}, \quad 0 \leq \gamma < 1.$$

La política $\pi(a|s)$ especifica la distribución de acciones que el agente puede seleccionar en un estado.

2.2. Componentes, definiciones, uso y justificación

Cuadro 2.1: Componentes funcionales del aprendizaje por refuerzo.

Componente	Definición	Uso	Justificación
Agente	Sistema que selecciona acciones.	Tomar decisiones frente al estado observado.	Es la entidad que aprende la estrategia de actuación.
Entorno	Sistema con el que interactúa el agente.	Generar transiciones y recompensas.	Define las consecuencias de las decisiones.
Estado s	Representación de la situación actual.	Condicionar la selección de acciones.	Sin estado no existe información contextual para decidir.
Acción a	Decisión disponible para el agente.	Modificar el estado del entorno.	Conecta la política del agente con las consecuencias observables.
Recompensa r	Señal numérica recibida tras actuar.	Guiar el aprendizaje.	Permite definir qué comportamientos son deseables sin proporcionar una etiqueta para cada decisión.
Política π	Regla o distribución que determina acciones a partir de estados.	Seleccionar acciones.	Es el comportamiento que el agente intenta mejorar.
Retorno G_t	Suma descontada de recompensas futuras.	Definir el objetivo a maximizar.	Permite valorar decisiones considerando consecuencias a corto y largo plazo.
Exploración	Prueba de acciones alternativas.	Descubrir estrategias potencialmente mejores.	Sin exploración, el agente puede quedarse con una política subóptima.
Explotación	Selección de acciones conocidas como buenas.	Utilizar el conocimiento aprendido.	Convierte la experiencia acumulada en comportamiento efectivo.
Episodio	Secuencia desde un estado inicial hasta una condición de finalización.	Organizar la experiencia de entrenamiento.	Facilita el cálculo de retornos y la comparación entre interacciones.

2.3. Flujo de programación en Python

```

1 import numpy as np
2 import random
3
4 Q = np.zeros((10, 2))
5 alpha = 0.1

```

```
6 gamma = 0.95
7 epsilon = 0.10
8
9 for episode in range(1000):
10     state = random.randrange(10)
11
12     for step in range(100):
13         # Exploracin / explotacin
14         if random.random() < epsilon:
15             action = random.randrange(2)
16         else:
17             action = int(np.argmax(Q[state]))
18
19         # Interaccin simplificada
20         next_state = min(9, max(0, state + (1 if action else -1)))
21         reward = 1 if next_state == 9 else -0.01
22
23         # Actualizacin Q-Learning
24         Q[state, action] += alpha * (
25             reward + gamma * np.max(Q[next_state])
26             - Q[state, action]
27         )
28
29         state = next_state
30
31     if state == 9:
32         break
```

2.4. Justificación de la interacción

A diferencia del aprendizaje supervisado, aquí no se entrega una etiqueta correcta para cada acción. La señal aparece como consecuencia de la interacción. Esta diferencia es fundamental: el problema consiste en aprender una **estrategia de decisión secuencial**, no solamente una función predictiva estática.

Capítulo 3

Aprendizaje supervisado: pipeline de datos, modelo y predicción



Figura 3.1: Representación alternativa del pipeline supervisado.

3.1. Fundamentos teóricos

Esta imagen enfatiza la separación entre los datos, la preparación, el algoritmo y la inferencia. Desde el punto de vista de ingeniería de software, esta separación permite construir pipelines reproducibles.

El concepto de pipeline puede formalizarse como composición:

$$f(x) = f_m(f_{m-1}(\cdots f_2(f_1(x)))).$$

En un sistema supervisado, las transformaciones deben ajustarse únicamente con los datos permitidos por el protocolo de entrenamiento. Por ello, herramientas como `Pipeline` de scikit-learn

son importantes.

3.2. Componentes, definiciones, uso y justificación

Cuadro 3.1: Componentes del pipeline supervisado.

Componente	Definición	Uso	Justificación
Entrada	Datos originales del problema.	Alimentar el sistema.	Define el espacio de información disponible para el modelo.
Preprocesador	Transformación parametrizable de los datos.	Preparar las variables.	Garantiza una representación consistente entre entrenamiento e inferencia.
Escalado	Transformación de magnitudes a una escala común.	Facilitar el aprendizaje de ciertos algoritmos.	Evita que diferencias de escala dominen distancias u optimización.
Selector	Mecanismo para conservar variables relevantes.	Reducir dimensionalidad.	Puede disminuir ruido y coste computacional.
Modelo	Algoritmo predictivo.	Aprender la relación entre X e y .	Es la parte que realiza la inferencia.
Pipeline	Composición ordenada de transformaciones y estimador.	Automatizar entrenamiento e inferencia.	Reduce discrepancias entre los datos utilizados para entrenar y los utilizados en producción.
Persistencia	Serialización del pipeline entrenado.	Reutilizar el modelo.	Permite separar entrenamiento de servicio de inferencia.

3.3. Implementación

```

1 from sklearn.pipeline import Pipeline
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.feature_selection import SelectFromModel
4 from sklearn.ensemble import ExtraTreesRegressor
5
6 selector = SelectFromModel(
7     ExtraTreesRegressor(n_estimators=100, random_state=42)
8 )
9
10 pipeline = Pipeline([
11     ("scaler", StandardScaler()),
12     ("selector", selector),
13     ("model", ExtraTreesRegressor(
14         n_estimators=300, random_state=42
15     ))
16 ])
17
18 pipeline.fit(X_train, y_train)
19 prediction = pipeline.predict(X_test)

```

Capítulo 4

Deep Learning: representación jerárquica y optimización

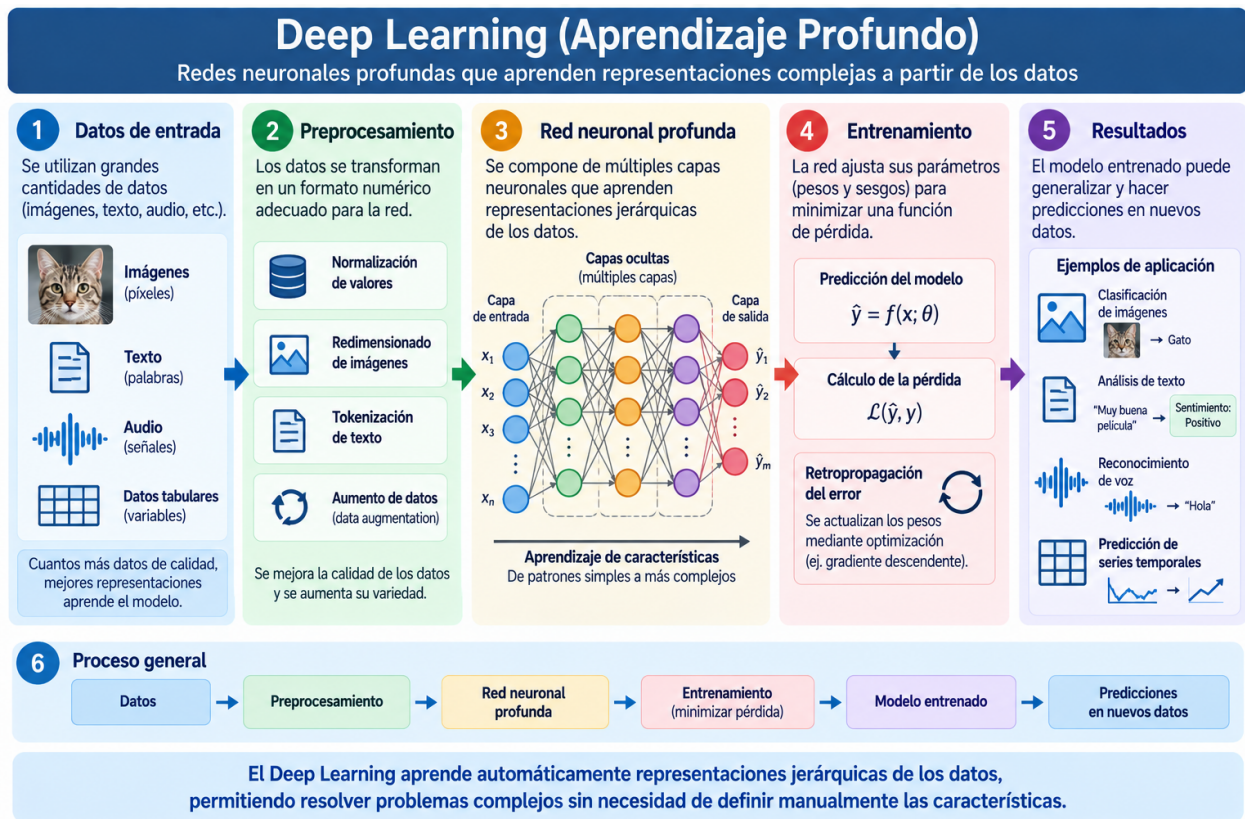


Figura 4.1: Proceso de Deep Learning.

4.1. Fundamentos teóricos

Deep Learning hace referencia a modelos basados habitualmente en redes neuronales con múltiples capas. Una red compone transformaciones sucesivas:

$$h^{(l)} = \sigma(W^{(l)}h^{(l-1)} + b^{(l)}).$$

La profundidad permite construir representaciones jerárquicas. En visión, por ejemplo, capas iniciales pueden detectar patrones locales y capas posteriores combinar esos patrones en estructuras de mayor nivel.

El entrenamiento busca minimizar:

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_i \ell(f_{\theta}(x_i), y_i).$$

La regla básica de actualización mediante descenso de gradiente es:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}.$$

4.2. Componentes, definiciones, uso y justificación

Cuadro 4.1: Componentes funcionales de Deep Learning.

Componente	Definición	Uso	Justificación
Datos	Ejemplos de entrada para la red.	Aprender representaciones.	Las redes profundas necesitan ejemplos suficientes para estimar numerosos parámetros.
Tensor	Estructura numérica multidimensional.	Representar datos y parámetros.	Las operaciones de las redes se expresan naturalmente mediante operaciones tensoriales.
Normalización	Transformación de las entradas a escalas adecuadas.	Estabilizar entrenamiento.	Puede facilitar la optimización y reducir problemas derivados de magnitudes incompatibles.
Capa neuronal	Transformación parametrizada de una representación.	Extraer o transformar características.	Permite construir funciones complejas mediante composición.
Función de activación	Función no lineal aplicada a la salida de una capa.	Introducir no linealidad.	Sin no linealidad, una composición de capas lineales seguiría siendo lineal.
Función de pérdida	Función que mide discrepancia entre predicción y objetivo.	Guiar la optimización.	Convierte el objetivo del aprendizaje en una magnitud optimizable.
Backpropagation	Cálculo eficiente de gradientes mediante regla de la cadena.	Obtener $\nabla_{\theta} \mathcal{L}$.	Permite actualizar millones de parámetros de forma computacionalmente viable.
Optimizador	Algoritmo que actualiza los parámetros usando gradientes.	Minimizar la pérdida.	Controla cómo se transforma el gradiente en cambios de parámetros.
Época	Una pasada completa por el conjunto de entrenamiento.	Organizar el entrenamiento.	Permite controlar y registrar la evolución del aprendizaje.
Generalización	Capacidad de funcionar sobre datos no utilizados durante el entrenamiento.	Evaluar el modelo.	El objetivo práctico no es memorizar los ejemplos sino aprender patrones transferibles.

4.3. Flujo de programación con PyTorch

```
1 import torch
2 from torch import nn
3
4 class Red(nn.Module):
5     def __init__(self, n_in, n_out):
6         super().__init__()
7         self.net = nn.Sequential(
8             nn.Linear(n_in, 128),
9             nn.ReLU(),
10            nn.Linear(128, 64),
11            nn.ReLU(),
12            nn.Linear(64, n_out)
13        )
14
15    def forward(self, x):
16        return self.net(x)
17
18 model = Red(10, 1)
19 optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
20 loss_fn = nn.MSELoss()
21
22 for epoch in range(100):
23     optimizer.zero_grad()
24     pred = model(X_train_tensor)
25     loss = loss_fn(pred, y_train_tensor)
26     loss.backward()
27     optimizer.step()
```

4.4. Relación entre profundidad y representación

La profundidad no debe interpretarse únicamente como “más capas”. Su utilidad procede de la posibilidad de aprender transformaciones jerárquicas y composicionales. La arquitectura debe seleccionarse según la naturaleza del dato: CNN para determinadas estructuras espaciales, modelos secuenciales para determinadas dependencias temporales o arquitecturas Transformer para relaciones mediante mecanismos de atención.

Capítulo 5

IA generativa: modelado de distribuciones y generación de contenido

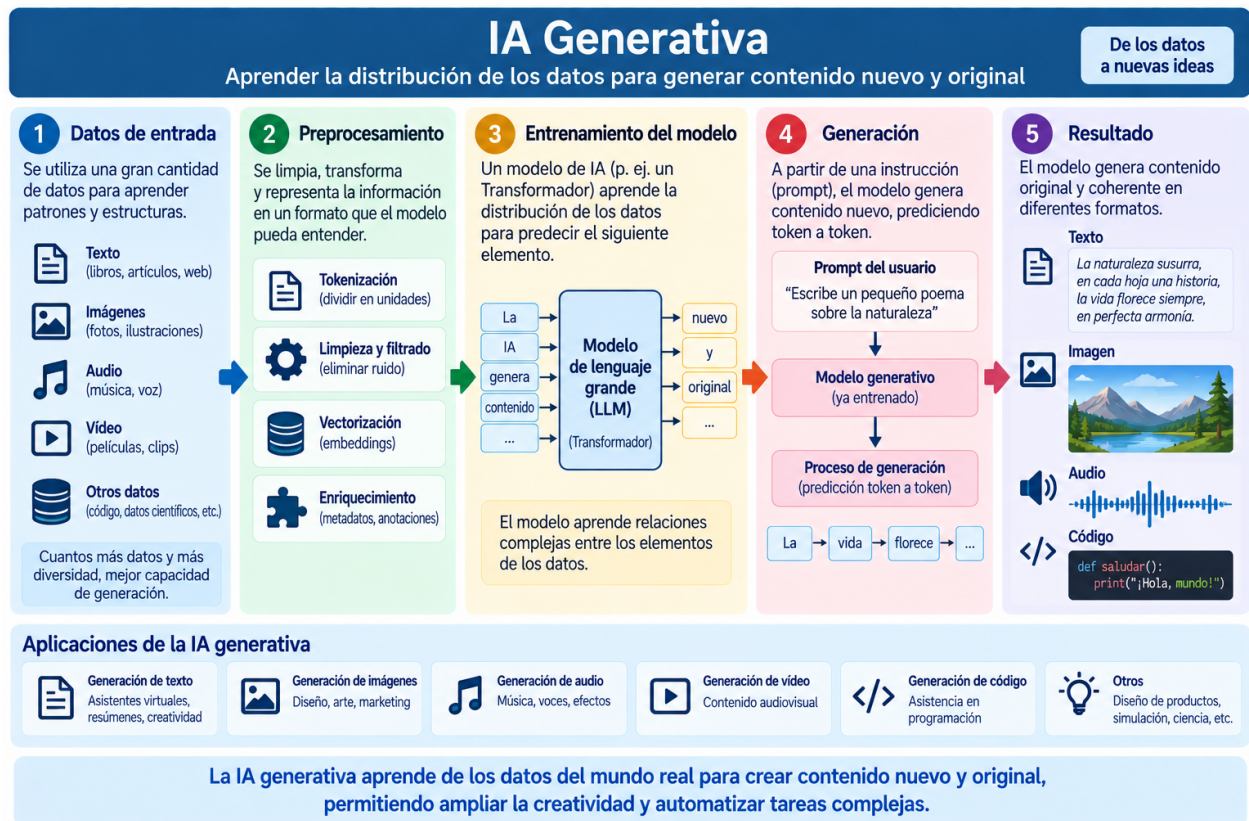


Figura 5.1: Pipeline conceptual de IA generativa.

5.1. Fundamentos teóricos

Un modelo generativo intenta aprender una distribución de probabilidad sobre los datos o una distribución condicionada. Para texto autoregresivo:

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t | x_1, \dots, x_{t-1}).$$

El entrenamiento busca que el modelo asigne alta probabilidad a secuencias observadas. En modelos de lenguaje, el objetivo típico es predecir el siguiente token.

Durante la generación, la distribución calculada por el modelo se convierte en una elección de token mediante una estrategia de decodificación. Por ello, generación y entrenamiento son fases diferentes.

5.2. Componentes, definiciones, uso y justificación

Cuadro 5.1: Componentes funcionales de la IA generativa.

Componente	Definición	Uso	Justificación
Corpus	Colección de datos utilizada para aprender patrones.	Entrenar el modelo.	La diversidad y calidad de los datos condicionan la distribución que puede aprenderse.
Tokenización	Conversión del contenido en unidades procesables.	Crear la secuencia de entrada del modelo.	El modelo opera sobre representaciones discretas o numéricas, no directamente sobre texto bruto.
Embedding	Representación vectorial de tokens u objetos.	Codificar relaciones semánticas o estructurales.	Facilita el procesamiento mediante álgebra vectorial.
Transformer	Arquitectura basada en mecanismos de atención y transformaciones de representaciones.	Modelar dependencias en secuencias.	Permite procesar relaciones entre elementos con gran capacidad de paralelización durante entrenamiento.
LLM	Modelo de lenguaje de gran escala.	Comprender y generar secuencias de texto.	Integra representaciones y parámetros suficientes para modelar relaciones lingüísticas complejas.
Prompt	Instrucción o contexto proporcionado durante la inferencia.	Condicionar la generación.	Permite especificar tarea, contexto, formato o restricciones.
Predicción del siguiente token	Estimación de $P(x_t x_{<t})$.	Generar secuencias autoregresivas.	Convierte el problema de generación en una sucesión de decisiones probabilísticas.
Decodificación	Proceso que selecciona tokens a partir de sus probabilidades.	Construir la respuesta final.	Controla diversidad, determinismo y comportamiento de generación.
Salida generada	Contenido producido por el modelo.	Resolver la tarea solicitada.	Es el resultado operativo del proceso generativo.
Evaluación y validación	Comprobación de calidad, coherencia y adecuación de la salida.	Controlar errores y riesgos.	Una salida plausible no garantiza que sea correcta o apropiada para el uso previsto.

5.3. Flujo de programación

```
1 from transformers import pipeline
2
3 generator = pipeline(
4     "text-generation",
5     model="distilgpt2"
6 )
7
8 prompt = "Explica qu es la inteligencia artificial:"
9 result = generator(
10     prompt,
11     max_new_tokens=80,
12     do_sample=True,
13     temperature=0.8
14 )
15
16 print(result[0]["generated_text"])
```

5.4. Aspectos conceptuales

La generación debe distinguirse de una base de datos que devuelve un registro existente. Un modelo generativo calcula una distribución condicionada y construye una secuencia según esa distribución.

En aplicaciones reales también son relevantes la recuperación de información, la validación de fuentes, los controles de seguridad, la privacidad, la evaluación de alucinaciones y la observabilidad del sistema.

Capítulo 6

IA simbólica: conocimiento explícito y razonamiento

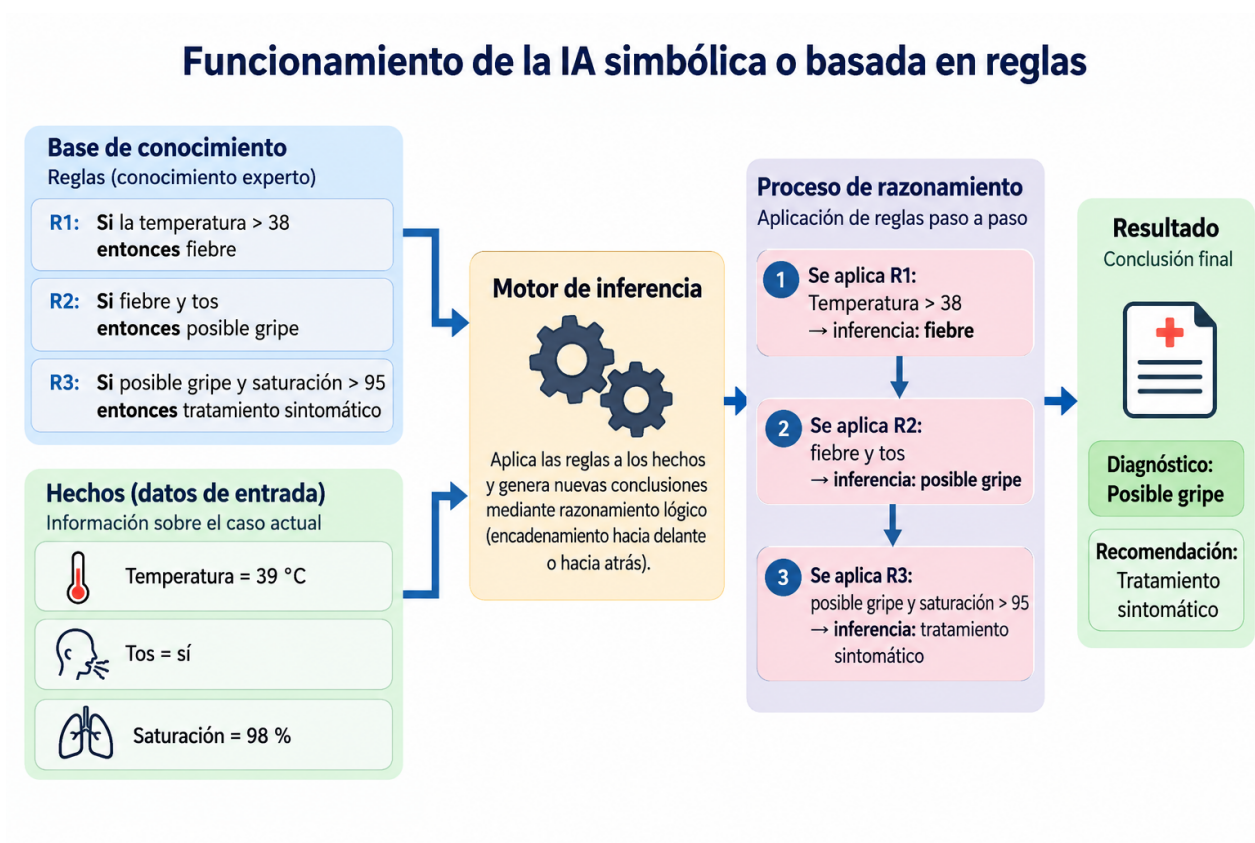


Figura 6.1: Funcionamiento de un sistema de IA simbólica basado en reglas.

6.1. Fundamentos teóricos

La IA simbólica representa explícitamente conocimiento mediante símbolos, hechos, reglas, ontologías o estructuras lógicas. Una regla puede escribirse como:

$$A \wedge B \Rightarrow C.$$

El sistema parte de hechos conocidos y aplica reglas para obtener consecuencias. En encadenamiento hacia delante, si las premisas de una regla son verdaderas, se añade su conclusión al conjunto de hechos.

Formalmente, si:

$$F = \{f_1, \dots, f_n\}$$

es el conjunto de hechos y R el conjunto de reglas, el motor busca obtener el cierre deductivo

$$F^* = \text{Cl}_R(F).$$

6.2. Componentes, definiciones, uso y justificación

Cuadro 6.1: Componentes funcionales de la IA simbólica.

Componente	Definición	Uso	Justificación
Hecho	Proposición o dato considerado verdadero dentro del sistema.	Representar el estado conocido del problema.	Constituye la evidencia sobre la que puede razonar el motor.
Regla	Implicación entre condiciones y una conclusión.	Derivar nuevo conocimiento.	Permite expresar conocimiento experto de manera explícita y auditable.
Base de conocimiento	Conjunto organizado de hechos, reglas y relaciones.	Centralizar el conocimiento del dominio.	Separa el conocimiento del mecanismo que lo procesa.
Motor de inferencia	Componente que determina qué reglas se activan y qué conclusiones producen.	Realizar razonamiento automático.	Convierte hechos y reglas en nuevas conclusiones.
Encadenamiento hacia delante	Razonamiento que parte de hechos disponibles y deriva consecuencias.	Resolver problemas orientados a datos.	Es apropiado cuando la llegada de nuevos hechos puede activar sucesivamente reglas.
Encadenamiento hacia atrás	Razonamiento que parte de una hipótesis y busca premisas que la sustenten.	Resolver consultas dirigidas a objetivos.	Reduce la búsqueda cuando existe una conclusión concreta que comprobar.
Memoria de trabajo	Conjunto de hechos actuales y hechos inferidos.	Mantener el estado del razonamiento.	Permite que conclusiones intermedias activen reglas posteriores.
Explicabilidad	Capacidad de mostrar qué reglas produjeron una conclusión.	Auditar el razonamiento.	Una regla explícita puede trazarse desde premisas hasta conclusión.

6.3. Flujo de programación

```

1 rules = [
2     ("R1", lambda f: f["temperature"] > 38, "fever"),
3     ("R2", lambda f: f.get("fever", False)
4         and f["cough"], "possible_flu"),
5     ("R3", lambda f: f.get("possible_flu", False)
6         and f["oxygen"] > 95,
7     "symptomatic_treatment")

```

```
8 ]
9
10 facts = {
11     "temperature": 39,
12     "cough": True,
13     "oxygen": 98
14 }
15
16 changed = True
17 while changed:
18     changed = False
19
20     for name, condition, conclusion in rules:
21         if condition(facts) and not facts.get(conclusion, False):
22             facts[conclusion] = True
23             changed = True
24             print(name, "=>", conclusion)
25
26 print(facts)
```

6.4. Ventajas y límites conceptuales

La representación explícita facilita la trazabilidad y permite modificar reglas sin reentrenar un modelo estadístico. Sin embargo, mantener grandes bases de reglas puede resultar costoso; además, el razonamiento clásico requiere representar adecuadamente incertidumbre, excepciones, ambigüedad y conocimiento incompleto cuando el dominio lo exige.

Nota

El ejemplo sanitario de la imagen debe entenderse como una demostración de razonamiento simbólico. No constituye por sí mismo un procedimiento médico válido.

Capítulo 7

Relación entre los paradigmas

7.1. Comparación conceptual

Cuadro 7.1: Comparación de los cinco paradigmas representados en las imágenes.

Paradigma	Fuente del conocimiento	Mecanismo	Unidad de aprendizaje	Justificación del enfoque
Supervisado	Ejemplos etiquetados	Optimización de una función de pérdida	Relación entrada-salida	Adecuado cuando existen ejemplos donde se conoce el resultado esperado.
Refuerzo	Experiencia y recompensas	Optimización de retorno/política	Decisiones secuenciales	Adecuado cuando las acciones modifican el entorno y sus consecuencias.
Deep Learning	Grandes conjuntos de datos	Optimización de redes profundas	Representaciones jerárquicas	Útil cuando la representación de características es compleja y puede aprenderse automáticamente.
Generativo	Distribución de datos	Modelado probabilístico/representacional	Patrones de generación	Permite producir nuevas instancias condicionadas por contexto.
Simbólico	Conocimiento explícito	Inferencia lógica/reglas	Hechos y relaciones	Útil cuando el conocimiento puede expresarse explícitamente y se requiere trazabilidad.

7.2. Patrón común de ingeniería

Aunque los mecanismos son diferentes, un sistema de IA suele contener:

Entrada → Representación → Procesamiento → Salida → Evaluación.

La diferencia fundamental está en cómo se obtiene el conocimiento:

- supervisado: a partir de ejemplos con respuesta;
- refuerzo: a partir de interacción y recompensa;
- Deep Learning: mediante representación jerárquica aprendida;
- generativo: mediante modelado de distribuciones y generación;
- simbólico: mediante conocimiento explícito y reglas.

7.3. Arquitectura Python orientativa

```
1 proyecto_ia/  
2 |-- data/  
3 |-- models/  
4 |-- src/  
5 | |-- data.py  
6 | |-- preprocessing.py  
7 | |-- supervised.py  
8 | |-- reinforcement.py  
9 | |-- deep_learning.py  
10 | |-- generative.py  
11 | '-- symbolic.py  
12 |-- notebooks/  
13 '-- main.py
```

7.4. Conclusión

Las imágenes pueden utilizarse como mapas conceptuales de cinco grandes mecanismos de IA. La ampliación teórica permite pasar de una descripción visual de “qué ocurre” a una explicación de “por qué ocurre”, “qué componente lo realiza”, “qué información necesita” y “cómo puede implementarse”.

El formato `longtable` permite además que las tablas de componentes se dividan automáticamente entre páginas, algo especialmente útil cuando el documento se utiliza como material docente o como base para una unidad didáctica extensa.

Apéndice A

Recursos y repositorios de aprendizaje

Este apéndice reúne recursos externos relacionados con los contenidos desarrollados en los capítulos anteriores. Se incluyen tanto recursos de recopilación como repositorios de código y cursos prácticos. Las direcciones se presentan mediante enlaces clicables para facilitar su consulta desde la versión PDF.

A.1. Repositorio de recursos sobre Inteligencia Artificial

A.1.1. Mejores repositorios GitHub para aprender IA

El artículo de Pasquale Pillitteri recopila repositorios de GitHub orientados al aprendizaje de Inteligencia Artificial. Puede utilizarse como punto de partida para ampliar la colección de materiales prácticos y localizar proyectos relacionados con Machine Learning, Deep Learning, IA generativa y otros ámbitos.

- **Recurso:** mejores repositorios GitHub para aprender IA.
- **Tipo:** artículo / recopilación de recursos.
- **Uso didáctico:** localizar repositorios adicionales para ejercicios, proyectos y autoaprendizaje.
- **URL:** <https://pasqualepillitteri.it/es/news/7249/mejores-repositorios-github-aprender-ia>

A.2. Microsoft – Generative AI for Beginners

El repositorio *Generative AI for Beginners* de Microsoft proporciona un itinerario de aprendizaje centrado en los fundamentos y construcción de aplicaciones de IA generativa. Actualmente organiza el material en 21 lecciones e incluye explicaciones conceptuales y ejemplos de código, con materiales en Python y TypeScript cuando corresponde. También dispone de una traducción al español. [2]

- **Organización:** Microsoft.
- **Contenido:** fundamentos de IA generativa, LLM, prompting y construcción de aplicaciones.
- **Formato:** lecciones, documentación y ejemplos de código.
- **URL:** <https://github.com/microsoft/generative-ai-for-beginners>
- **Versión en español:** <https://github.com/microsoft/generative-ai-for-beginners/tree/main/translations/es>

El repositorio resulta especialmente relacionado con el capítulo de IA generativa de este documento, ya que desarrolla contenidos sobre modelos de lenguaje y construcción de aplicaciones. El propio repositorio describe un itinerario de 21 lecciones y proporciona ejemplos de Python y TypeScript. [2]

A.3. Curso Básico de IA – Eneas Carneiro

El repositorio *Curso-Basico-IA* puede utilizarse como recurso complementario para ampliar los contenidos introductorios de Inteligencia Artificial y desarrollar actividades prácticas basadas en GitHub.

- **Autor/organización indicada en el repositorio:** Eneas Carneiro.
- **Tipo:** repositorio educativo.
- **Uso didáctico:** material complementario para ejercicios y aprendizaje práctico.
- **URL:** <https://github.com/eneascarneiro/Curso-Basico-IA>

A.4. Cursos de Python – Instituto Humai

El repositorio del Instituto Humai agrupa cursos relacionados con Python, ciencia de datos, aprendizaje automático y herramientas de programación. Incluye contenidos de introducción a Python, análisis de datos, automatización y web scraping, APIs, bases de datos y SQL. [4]

- **Organización:** Instituto Humai.
- **Contenido:** Python, análisis de datos, automatización, APIs, bases de datos, SQL y contenidos relacionados con IA y ciencia de datos.
- **Uso didáctico:** preparación de la base de programación necesaria para implementar los flujos Python de los capítulos anteriores.
- **URL:** <https://github.com/institutohumai/cursos-python>

A.5. Mapa de correspondencia con los capítulos

Cuadro A.1: Relación entre los capítulos y los recursos externos.

Capítulo	Recurso relacionado	Aplicación didáctica
Aprendizaje supervisado	Recopilaciones GitHub de IA y recursos de Python	Buscar notebooks y proyectos para clasificación, regresión y evaluación de modelos.
Aprendizaje por refuerzo	Repositorios educativos de IA	Localizar implementaciones experimentales de agentes y entornos.
Pipeline supervisado	Cursos de Python y ciencia de datos de Humai	Reforzar estructuras de datos, funciones, módulos y procesamiento de datos.
Deep Learning	Recursos de aprendizaje de IA y repositorios GitHub	Ampliar redes neuronales, entrenamiento y experimentación.
IA generativa	Microsoft Generative AI for Beginners	Estudiar LLM, prompting y construcción de aplicaciones generativas.

Capítulo	Recurso relacionado	Aplicación didáctica
IA simbólica	Repositorios generales de IA	Experimentar con representación del conocimiento, reglas y motores de inferencia.

A.6. Referencias web

Bibliografía

- [1] Pillitteri, P. *Mejores repositorios GitHub para aprender IA*. Disponible en: <https://pasqualepillitteri.it/es/news/7249/mejores-repositorios-github-aprender-ia>
- [2] Microsoft. *Generative AI for Beginners*. Repositorio GitHub, 21 lecciones sobre fundamentos y desarrollo de aplicaciones de IA generativa. Disponible en: <https://github.com/microsoft/generative-ai-for-beginners>
- [3] Carneiro, E. *Curso-Basico-IA*. Repositorio GitHub. Disponible en: <https://github.com/eneascarneiro/Curso-Basico-IA>
- [4] Instituto Humai. *cursos-python*. Repositorio GitHub con cursos de Python, análisis de datos, automatización, APIs, bases de datos y contenidos relacionados con IA y ciencia de datos. Disponible en: <https://github.com/institutohumai/cursos-python>